FOUR-COURSE SYSTEM

COURSE 3 - DATA ENGINEERING 2026

C3-110 Capstone Architecture Brief

RetailPulse production capstone: problem, NFRs, boundaries, architecture and acceptance evidence

Status	RC1 - FINAL-LAYER QA PASS CANDIDATE
Course boundary	C3-104 = C3-09 phase Gate only; C3-113 = separate course-wide graduation Gate
Release boundary	Course 3 is NOT FINAL until Mentor/PWA + whole-course release regression are complete.

Permanent engineering behavior controls the course. Vendor UI/version notes are refreshable references, and external links are support-only; the course artifacts remain sufficient without them.

1. Why this artifact exists

RetailPulse is the coherent production-style system carried forward from C3-09. C3-110 freezes the business and architecture contract that the separate C3-113 graduation Gate will test under an unseen incident and a fresh controlled change. It does not replace your learner-owned ADR; it defines what that ADR must prove.

RetailPulse baseline controls

Base orders	5
Incremental change records	3
Trusted current orders after deterministic merge	7
O102 current amount	1050.00
O106 / O107	inserted once each
Completed revenue	6910.00

2. Business and non-functional requirements

Freshness	Daily commerce product available within 3 hours of declared source cutoff.
Corrections	Late/new changes update current state by order_id without duplicate business effects.
Backfill	Historical dates can be reprocessed independently using the same validation controls.
Trust	Critical quality or reconciliation failure blocks trusted publish.
Security	Analysts consume curated data; raw/customer-sensitive evidence stays restricted and sanitized.
Recovery	A failed boundary can be contained and replayed without redoing unrelated successful work.
Handoff	Another engineer can reproduce the required run from repository + documented environment + runbook.

3. Picture the architecture before choosing tools

Source contracts → immutable/source-preserving landing → validated current-state layer → engineered transforms/Spark component → serving product → consumer. Orchestration coordinates boundaries; quality/reconciliation can block promotion; observability records run/diagnostic evidence; security limits identities; version control/CI controls changes.

Mandatory capability map

Ingestion	Base + incremental/change handling; explicit source contract; replay behavior.
Storage	Landing/Bronze, validated Silver/current state, serving/Gold contract.
Transformation	Reusable SQL/Python logic, tests, and at least one real Spark component if Spark is claimed.
Orchestration	Dependencies, bounded retries, partial rerun, and one historical backfill/reprocessing path.
Quality	Blocking checks + key/count/value reconciliation + quarantine/reject behavior.
Observability	Run IDs, structured logs/events, SLI/SLO reasoning, alert/diagnostic evidence.
Security	Secrets outside repository, least privilege, privacy-safe evidence, consumer boundary.
Lifecycle	Version control, automated validation, controlled promotion, rollback/roll-forward and post-change checks.

4. Streaming decision

Streaming is optional. Include it only when your ADR names a credible low-latency consumer and defines keys, schema compatibility, offsets/checkpoints, replay and failure recovery. A batch-first design is correct when the daily freshness requirement is sufficient.

5. Entry evidence required before C3-113

• A clean end-to-end baseline run from declared inputs to serving output with retained run evidence.

• Baseline merge reconciles to 7 current orders, O102=1050.00, O106/O107 once each, completed revenue=6910.00.

• One independent replay/backfill path demonstrated safely.

• Blocking quality/reconciliation prevents unsafe trusted publish.

• ADR, source contracts, README, runbook and evidence index are reproducible and sanitized.

• Every runtime/platform claim has real prior execution evidence; conceptual knowledge is not relabeled as executed proof.

6. Architecture defense prompts

• Why is each component present? What simpler design did you reject and why?

• Where is idempotence enforced? Where could duplicates still enter?

• Which failure can be retried automatically, and which must stay failed until repair?

• What changes first at 10x volume? What evidence supports your answer?

• Which design patterns transfer if Fabric is replaced by AWS/GCP/another platform?